

Ride Booking Application

Use Case

A ride booking application offers different types of rides such as **Mini**, **Sedan**, and **SUV**. Each ride type follows a different fare calculation policy based on the distance traveled.

Objective

To design a ride booking system using **Object-Oriented Programming (OOP)** concepts where:

- A common base class represents all rides
- Fare calculation is customized using **method overriding**
- **Polymorphism** is used to calculate fare through a base class reference

Class Structure Overview

Base Class: Ride

- The Ride class acts as the **parent class**.
- It stores common data such as **distance**.
- It defines a default `calculate_fare()` method.
- This method provides a common interface for all ride types.

Derived Classes (Inherited Classes)

MiniRide

- Inherits from the Ride class.
- Overrides the `calculate_fare()` method.
- Calculates fare using a **lower base fare and per-kilometer charge**.

SedanRide

- Inherits from the Ride class.
- Overrides the `calculate_fare()` method.
- Uses a **moderate base fare and per-kilometer charge**.

SUVRide

- Inherits from the Ride class.
- Overrides the `calculate_fare()` method.
- Applies a **higher base fare and per-kilometer charge** due to premium service.

Method Overriding

- The method `calculate_fare()` is defined in the base class and redefined in each child class.
 - Each ride type provides its **own fare calculation logic**.
 - This allows different behaviors while using the same method name.
-

Polymorphism in Fare Calculation

- A function `show_fare()` accepts a **Ride type reference**.
- At runtime, this reference can point to **MiniRide**, **SedanRide**, or **SUVRide** objects.
- When `calculate_fare` is called, Python automatically executes the **correct overridden method**.
- This demonstrates **runtime polymorphism**.

Program Flow Explanation

1. The user selects a ride type (Mini, Sedan, or SUV).
2. The user enters the travel distance.
3. Based on the selection, an appropriate ride object is created.
4. The object is passed to a function expecting a Ride reference.
5. Fare is calculated using polymorphism.
6. The final fare is displayed to the user.

Key OOP Concepts Used

- **Inheritance** – Child ride classes inherit common features from the Ride class.
- **Method Overriding** – Each ride type customizes fare calculation.
- **Polymorphism** – A single base class reference is used to handle multiple ride types.
- **Abstraction** – Internal fare logic is hidden from the user.

Ride Booking Application Design

This design implements a ride-hailing system that uses **Runtime Polymorphism** to calculate fares based on vehicle types.

1. Data Flow Diagram (DFD) Structure

A **Level 0 (Context Diagram)** shows the overall system's interactions with external entities.

- **Entities:** User (Passenger), Driver, and Database.
- **Data Flows:**
 - **User → System:** Login credentials, ride request (source, destination, vehicle type).
 - **System → User:** Fare estimates, ride confirmation, driver details.
 - **System ↔ Database:** Store/retrieve booking records and pricing constants.
 - **System → Driver:** Ride notification and passenger location.

2. Class Implementation (Python)

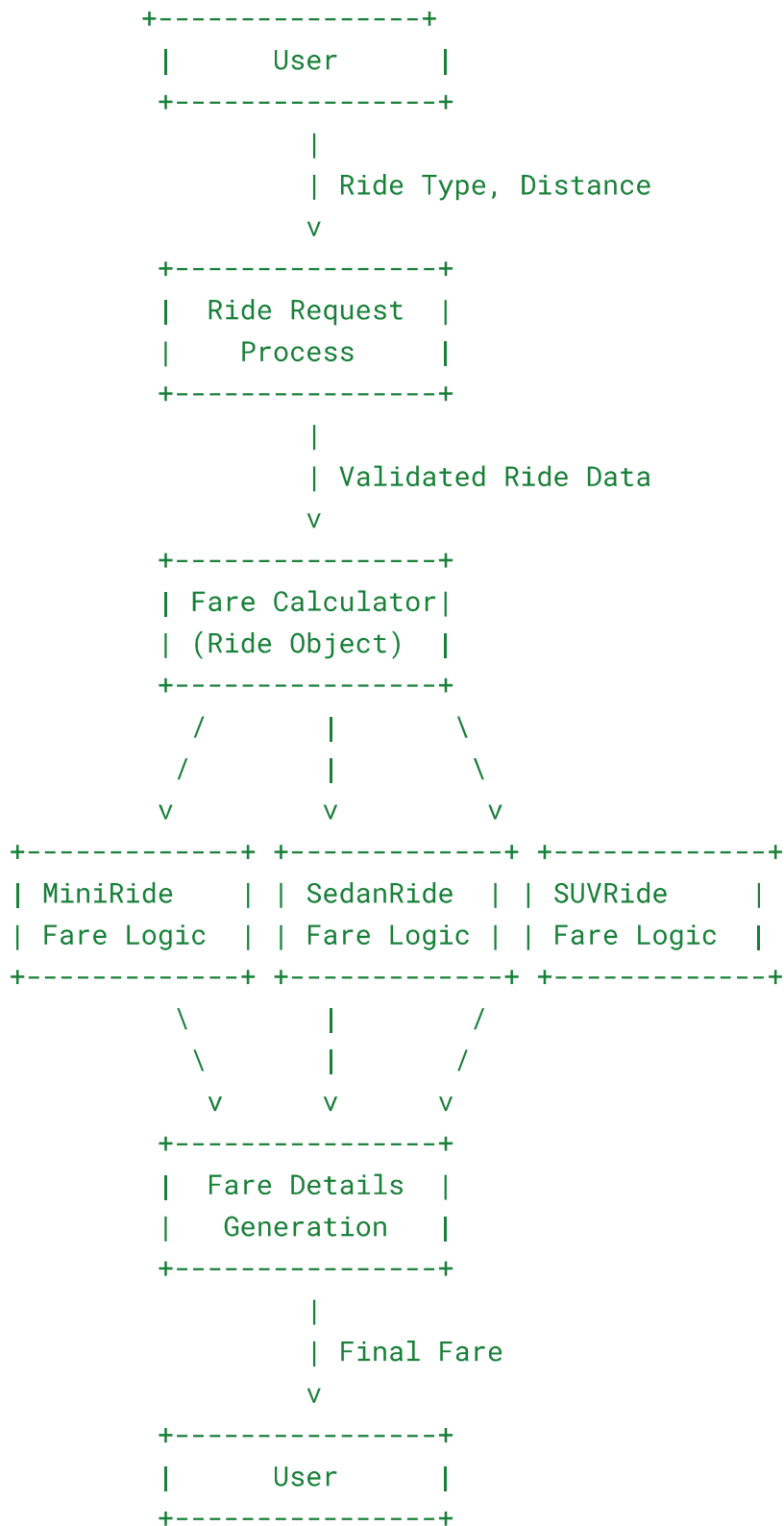
The implementation uses a base `Ride` class with specialized child classes that override the `calculate_fare` method.

3. Key Concepts Explained

- **Inheritance:** `MiniRide`, `SedanRide`, and `SUVRide` inherit common attributes (like distance) from the `Ride` class.
- **Method Overriding:** Each child class provides its own logic for `calculate_fare(distance)`, allowing specific pricing for different car models.

- **Polymorphism:** A single `Ride` reference can point to any of its subclasses. At runtime, the system automatically determines which fare logic to apply based on the object type.
- **System Design:** The system manages the lifecycle from request to completion, including fare estimation, driver matching, and payment processing.

DFD – Diagram



FLOWCHART FOR RIDE BOOKING APPLICATION

